

IDDQ Labs :-

IDDQ: current drawn by CMOS device in quiescent state (stable state $\rightarrow$ i/p are stable)

IDD $\rightarrow$ current that flows from VDD to Gnd in a CMOS circuit

Q $\rightarrow$ quiescent

We apply certain test pattern and we wait for certain amount of time so that the circuit becomes stable. After this we measure IDDQ.
 $\hookrightarrow$ After manufacturing.

Good CMOS circuit $\rightarrow$ very low IDDQ

Defective CMOS circuit $\rightarrow$ high IDDQ.

labc :-

$f_{n1} \Rightarrow$ this rule determines if a module net is floating.

$f_{n3} \Rightarrow$ this rule determines if an instance net is floating.

line no $\Rightarrow$ 27, 28 $\Rightarrow$ we are making these 2 rules as errors

$\Downarrow$

If any of the rule fails, we can proceed with our run only after fixing the rule failures.

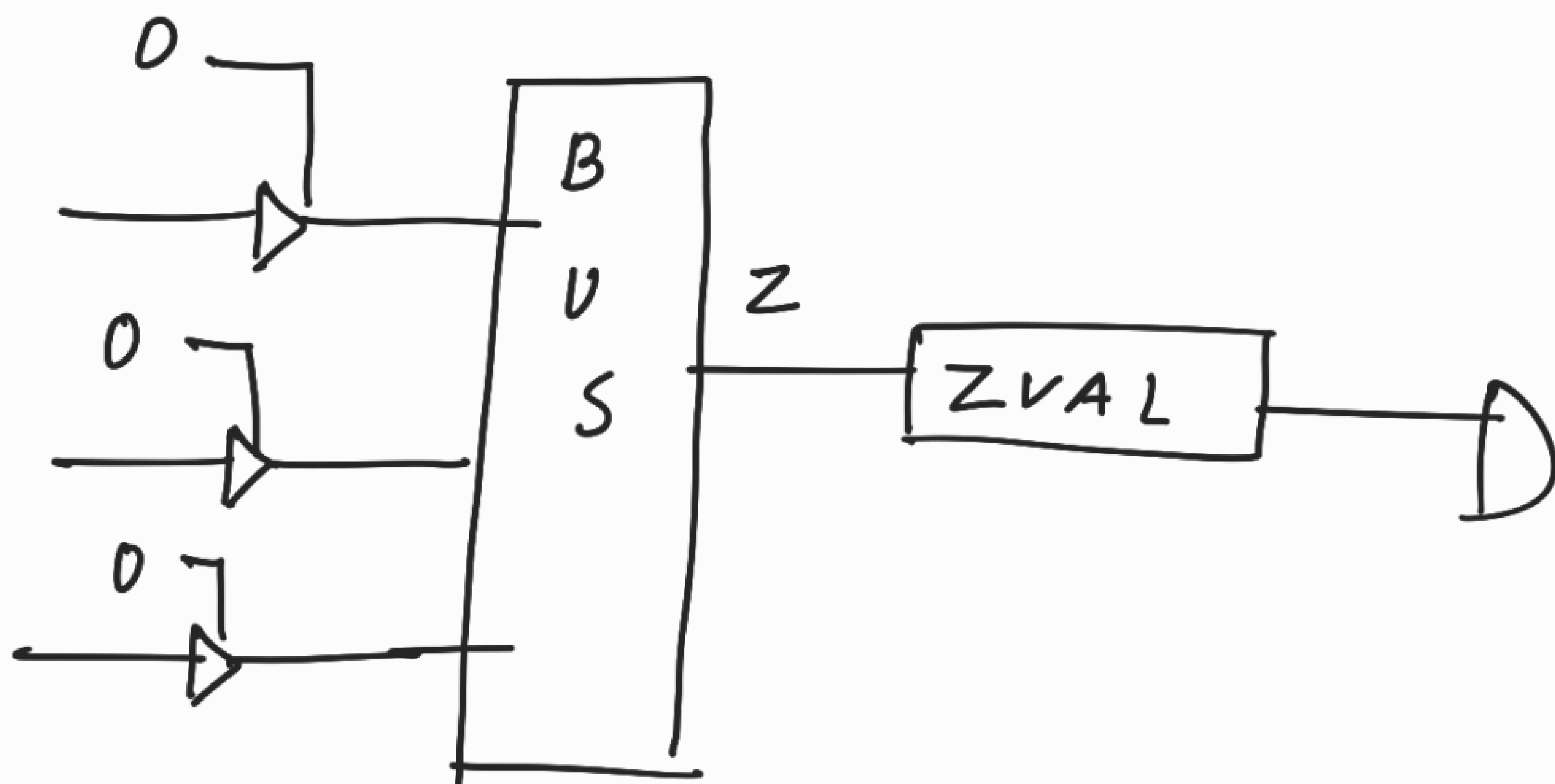
To ensure that no current flows in the quiescent state, every node must be pulled either low or high, and not allowed to float. For example, if the input of the inverter is allowed to float, the voltage could drift to an intermediate value, putting both transistors into a partially on state. This would allow a steady-state current to flow from VDD through the two transistors to ground.

* IDDq testing is a method to detect defects like stuck on, bridging in circuits by measuring IDDq. A good circuit should have very low IDDq. The below checks are essential to ensure that a good circuit does not raise IDDq current during measurement.

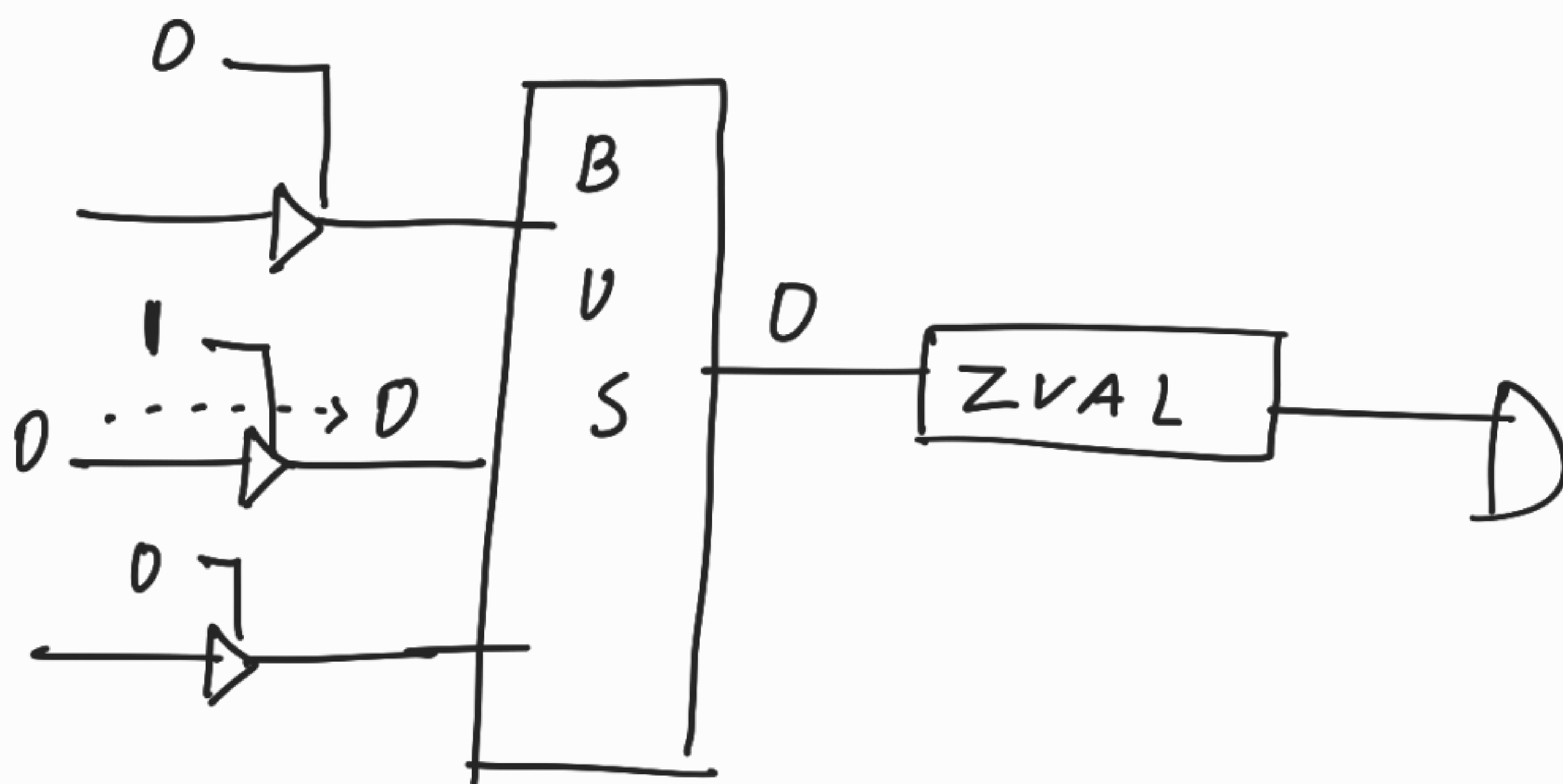
line no 59:- $\rightarrow$ Specifies the restrictions & conditions to use when creating or selecting patterns
set-iddq-checks -int-float -bus -clock see * above

- bus: An optional switch that specifies not to allow contention condition on bus gates
- clock: An optional switch that specifies to not allow clock pins to be on during IDDQ measure
- int-float :- An optional switch that specifies to not allow a floating (z) value on any ZVAL primitive gate that drives any boolean logic.

Note :- A ZVAL gate is placed b/w a source (z producer) and a sink (non z consumer)



This type of pattern should not be written by the tool.



Tool has to ensure that any one of the tristate is enabled.

line no. 63:-

analyze-restriction:-

Performs an analysis to automatically determine source of problem from a failed ATPG run.

NOTE 1:- Coverage Analysis

If SA COV is good, then automatically IDDQ COV will also be good.

Reason:-

Test pattern IDDQ: Test set activate SA fault but its effect need not propagate to any obs point.

Propagation to obs point is not required because IDDQ can be measured by current.

i.e; in IDDQ we are not going to check the value at the O/P. We are only going to check whether the cell draws excessive current in its quiescent state or not.

patterns for SA :- tool will do activation
+
propagation to obs pt.

patterns for IDDQ :- tool will only activate the fault site.

If SA cov is good $\Rightarrow$ tool was able to activate most of the fault sites.



Obviously IDDQ cov will also be good.

NOTE 2 :- During IDDQ, we activate a logic by applying patterns, wait for certain amount of time for the circuit to become stable and then measure IDDQ.

Good - low IDDQ

Defective - High IDDQ.

* Tool will write the patterns ^{specifically} to activate the logics by using EDT.

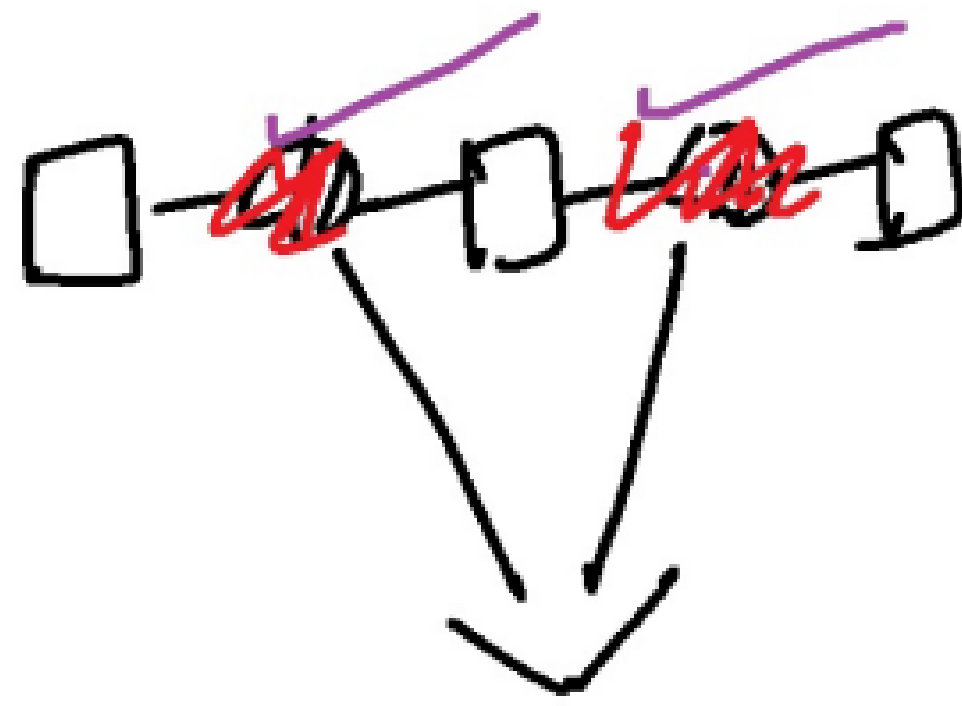
So the tool feels that it is able to activate those logics. Hence it places the faults in those logics in the detected class.

The tool is not specifically trying to activate EDT logic. So the tool might feel that it has not activated the EDT logic. So it considers the EDT faults as not detected.

But logically if we see, in the process of applying patterns to activate the logic, EDT also automatically gets activated. So if there are any man-defects in EDT logic (stuck on, bridging), it will also reflect in the IDDQ current in silicon & will also get detected.

So we can ignore the EDT faults.

Extra Classroom Paint Explanations



during 100a pattern gen run,
tool will specifically write
patterns to activate these logic

How tool gen those patterns?
↳ using EDT logic.

decomp (part of
EDT logic) gen
unique seq
that has to
be applied
onto each
scan char

NOTE:- EDT topic

decomp the value
a ik up and it
will apply a
unique seq in each
of the scan char

→ So the tool feels that it has activated those logic (the tool feels that it has gen patterns to activate those logic)
∴ The tool places the faults in those sites under det class during fault sim process.

↳ The tool is assuming that if any stuck on / bridging kind of faulty beh occurs after mon due to the presence of defect, it will get det with the gen test pattern

But the tool has not written any pattern specifically to activate [A]

EDT logic.

↳ Because EDT logic is used for pattern gen.

So the tool might be feeling that it has not written any pattern to activate EDT logic.

So the tool places the EDT logic faults under not det class

(AU class) during fault sim.

LOGICALLY: In the process of applying ^{/gen} patterns to activate the logic, (i.e. ② ③ of scan comp logic)

EDT logic also automatically gets activated.

↓
If there is any major defect in EDT logic (stuck on, bridging), it will reflect ^{also} in the IDDQ current in Silicon. ⇒ It will get detected due to the low drop ~~pattern~~ ^{EDT faults in IDDQ pat gen run.}